



TECNOLÓGICO
NACIONAL DE MÉXICO



**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO SUPERIOR DE
MISANTLA**

ACTIVIDAD:

Reporte de Ejercicios en google colab

ASIGNATURA:

Taller de Base de Datos

DOCENTE:

Dr. Irahan Otoniel José Guzmán

ALUMNA:

Alexandra Julieth Sandoval Dorantes

UNIDAD: II

CARRERA:

Ingeniería en Sistemas Computacionales

SEMESTRE: 1

GRUPO:503A

FECHA DE ENTREGA: 16-09-2026

ÍNDICE

Introducción.....	2
Ejercicio 1 — Comida (restaurante).....	3
Conexión a la base de datos	3
Creación de la tabla	3
Inserción de registros	4
Modificación de registros	5
Eliminación de registros	5
Ejercicio 2 — Alumnos (escuela)	7
Conexión a la base de datos	7
Creación de la tabla	7
Inserción de registros	8
Modificación de registros	8
Eliminación de registros	9
Ejercicio 3 — Tiendita escolar (productos)	10
Conexión a la base de datos	10
Creación de la tabla	10
Inserción de registros	11
Modificación de registros	12
Eliminación de registros	12
Conclusiones.....	13

Introducción

El presente reporte muestra el desarrollo de tres ejercicios prácticos sobre creación y manipulación de tablas en una base de datos, utilizando el lenguaje SQL dentro de un cuaderno de Google Colab. En cada ejercicio se llevó a cabo el mismo proceso: primero se definió la estructura de una tabla mediante la instrucción `CREATE TABLE`, después se agregaron registros de ejemplo con `INSERT`, posteriormente se modificaron algunos de esos registros utilizando `UPDATE` y, finalmente, se eliminaron los que ya no cumplían con las condiciones establecidas mediante `DELETE`. Para ejecutar y comprobar cada instrucción se utilizó la librería `sqlite3` de Python, junto con dos funciones auxiliares (`query` y `run`) que permiten correr comandos SQL y visualizar los resultados en forma de tabla.

Ejercicio 1 — Comida (restaurante)

Este primer ejercicio consistió en registrar, en una base de datos, el menú de platillos que ofrece una cadena de restaurantes. Para ello se creó una tabla llamada comida, con un identificador único para cada platillo, su nombre, la categoría a la que pertenece (entrada, plato fuerte, postre o bebida), su precio y si se encuentra o no disponible actualmente.

Conexión a la base de datos

Antes de crear cualquier tabla, es necesario preparar el entorno de trabajo. En esta primera celda se importan las librerías sqlite3 y pandas, se abre una conexión en memoria hacia una base de datos temporal y se define un cursor, que es el objeto encargado de ejecutar los comandos SQL. También se crean dos funciones de apoyo: query, para consultar información, y run, para insertar, modificar o eliminar registros.

```
[2]
✓ 0 s

import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 1. Conexión a la base de datos y definición de funciones auxiliares.

Creación de la tabla

A continuación se crea la tabla comida, definiendo el tipo de dato y las restricciones que debe cumplir cada campo: el identificador del platillo es autoincremental, el nombre y el precio no pueden quedar vacíos, y el campo de disponibilidad queda en verdadero de manera predeterminada si no se especifica lo contrario.

```
[3]
✓ Os
cur.executescript('''
DROP TABLE IF EXISTS comida;

CREATE TABLE comida (
    id_platillo INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    categoria VARCHAR(30),
    precio DECIMAL(6,2) NOT NULL,
    disponible BOOLEAN DEFAULT TRUE
);
''')
conn.commit()
print("Tabla creada: comida")

▼ ... Tabla creada: comida
```

Figura 2. Creación de la tabla comida con sus campos y restricciones.

Inserción de registros

Una vez creada la tabla, se agregaron ocho platillos distintos, distribuidos en las cuatro categorías solicitadas. En el caso del Flan napolitano, se marcó intencionalmente como no disponible para más adelante poder mostrar el funcionamiento de la eliminación de registros.

```
s
# INSERT: alta de platillos
run("""INSERT INTO comida (nombre, categoria, precio, disponible) VALUES
('Ensalada', 'entrada', 65.00, TRUE),
('Sopa de tortilla', 'entrada', 55.00, TRUE),
('Milanesa de pollo', 'plato fuerte', 120.00, TRUE),
('Tacos al pastor', 'plato fuerte', 95.00, TRUE),
('Pastel de chocolate', 'postre', 45.00, TRUE),
('Flan napolitano', 'postre', 40.00, FALSE),
('Limonada', 'bebida', 30.00, TRUE),
('Chokomilk', 'bebida', 25.00, TRUE);""")

query("SELECT * FROM comida")
```

Figura 3. Inserción de los platillos en la tabla comida.

... OK -> INSERT INTO comida (nombre, categoria, precio, disponible) VALUES ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada	entrada	65	1
1	2	Sopa de tortilla	entrada	55	1
2	3	Milanesa de pollo	plato fuerte	120	1
3	4	Tacos al pastor	plato fuerte	95	1
4	5	Pastel de chocolate	postre	45	1
5	6	Flan napolitano	postre	40	0
6	7	Limonada	bebida	30	1
7	8	Chokomilk	bebida	25	1

Figura 4. Verificación de los platillos insertados.

Modificación de registros

Después se realizó un aumento del diez por ciento al precio de los platillos pertenecientes a la categoría de plato fuerte, como ejemplo de actualización de información dentro de la base de datos.

UPDATE: aumento de precio a la categoría "plato fuerte"

```
run("UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'plato fuerte';")
query("SELECT id_platillo, nombre, categoria, precio FROM comida WHERE categoria = 'plato fuerte'")
```

... OK -> UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'plato fuerte'; ...

	id_platillo	nombre	categoria	precio
0	3	Milanesa de pollo	plato fuerte	132.0
1	4	Tacos al pastor	plato fuerte	104.5

Figura 5. Actualización del precio de los platillos de la categoría plato fuerte.

Eliminación de registros

Por último, se eliminaron de la tabla los platillos marcados como no disponibles, aprovechando precisamente la columna disponible que se había definido desde la creación de la tabla.

DELETE: eliminación de registros (se retira el platillo no disponible)
run("DELETE FROM comida WHERE disponible = FALSE;")
query("SELECT * FROM comida")

... OK -> DELETE FROM comida WHERE disponible = FALSE; ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Ensalada	entrada	65.0	1
1	2	Sopa de tortilla	entrada	55.0	1
2	3	Milanesa de pollo	plato fuerte	132.0	1
3	4	Tacos al pastor	plato fuerte	104.5	1
4	5	Pastel de chocolate	postre	45.0	1
5	7	Limonada	bebida	30.0	1
6	8	Chokomilk	bebida	25.0	1

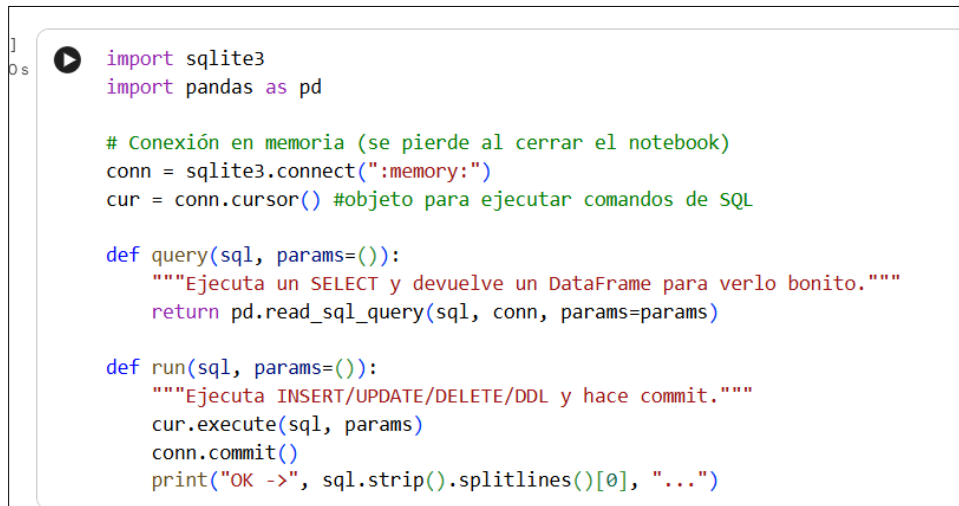
Figura 6. Eliminación del platillo marcado como no disponible.

Ejercicio 2 — Alumnos (escuela)

En este segundo ejercicio se registró la información básica de los alumnos de una escuela. Se creó una tabla llamada alumnos, que guarda el identificador de cada estudiante, su nombre, el grado que cursa, su edad y su promedio, este último con un valor por defecto de cero en caso de no especificarse.

Conexión a la base de datos

Se utilizó nuevamente la misma celda de conexión, ya que se trabaja sobre la misma base de datos en memoria, únicamente agregando una tabla nueva.



```
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 7. Conexión a la base de datos para el ejercicio de alumnos.

Creación de la tabla

Se definió la tabla alumnos con sus respectivos campos y restricciones: el identificador es autoincremental, el nombre y la edad son obligatorios, y el promedio queda en cero de manera predeterminada.

```
cur.executescript('''
DROP TABLE IF EXISTS alumnos;

CREATE TABLE alumnos (
    id_alumno INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    grado VARCHAR(10),
    edad INTEGER NOT NULL,
    promedio DECIMAL(4,2) DEFAULT 0.0
);
''')
conn.commit()
print("Tabla creada: alumnos")
```

Tabla creada: alumnos

Figura 8. Creación de la tabla alumnos con sus campos y restricciones.

Inserción de registros

Se dieron de alta tres alumnos de ejemplo, cada uno perteneciente a un grado distinto, con su edad y su promedio correspondiente.

```
# INSERT: alta de alumnos
run("""INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES
    ('Pedro Sánchez', '3ro A', 8, 85),
    ('Ana Torres', '4to B', 9, 89),
    ('Sofía Castillo', '5to B', 10, 95);""")

query("SELECT * FROM alumnos")
```

OK -> INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES ...

	id_alumno	nombre	grado	edad	promedio
0	1	Pedro Sánchez	3ro A	8	85
1	2	Ana Torres	4to B	9	89
2	3	Sofía Castillo	5to B	10	95

Figura 9. Inserción de los alumnos en la tabla alumnos.

Modificación de registros

Se aumentaron cinco puntos al promedio de los alumnos del grado 3ro A, como ejemplo de una actualización aplicada a un grupo específico dentro de la tabla.

```
[7]
✓ Os
# UPDATE: aumento de promedio al grado "3ro A"
run("UPDATE alumnos SET promedio = promedio + 5 WHERE grado = '3ro A';")
query("SELECT id_alumno, nombre, grado, promedio FROM alumnos WHERE grado = '3ro A'")

OK -> UPDATE alumnos SET promedio = promedio + 5 WHERE grado = '3ro A'; ...

  id_alumno  nombre  grado  promedio
0          1  Pedro Sánchez  3ro A      90
```

Figura 10. Actualización del promedio de los alumnos del grado 3ro A.

Eliminación de registros

Finalmente, se eliminó de la tabla al alumno perteneciente al grado 4to B, quedando únicamente los dos alumnos restantes.

```
[8]
✓ Os
# DELETE: eliminación de registros (se retira el alumno de "4to B")
run("DELETE FROM alumnos WHERE grado = '4to B';")
query("SELECT * FROM alumnos")

OK -> DELETE FROM alumnos WHERE grado = '4to B'; ...

  id_alumno  nombre  grado  edad  promedio
0          1  Pedro Sánchez  3ro A      8      90
1          3  Sofía Castillo  5to B     10      95
```

Figura 11. Eliminación del alumno del grado 4to B.

Ejercicio 3 — Tiendita escolar (productos)

El tercer ejercicio tuvo como propósito llevar el control de los productos que vende la tiendita de una escuela. Se creó una tabla llamada productos, que almacena el identificador de cada producto, su nombre, el tipo al que pertenece (snack, bebida, dulce o papelería), su precio y la cantidad disponible en existencia.

Conexión a la base de datos

Al igual que en los ejercicios anteriores, se utilizó la misma celda de conexión para trabajar sobre la base de datos en memoria.

```
[9]
✓ Os
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 12. Conexión a la base de datos para el ejercicio de productos.

Creación de la tabla

Se definió la tabla productos con sus campos y restricciones correspondientes: el identificador es autoincremental, el nombre y el precio son obligatorios, y la existencia queda en cero de manera predeterminada si no se especifica un valor.

```
cur.executescript('''
DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id_producto INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30),
    precio DECIMAL(6,2) NOT NULL,
    existencia INTEGER DEFAULT 0
);
''')
conn.commit()
print("Tabla creada: productos")
```

Tabla creada: productos

Figura 13. Creación de la tabla productos con sus campos y restricciones.

Inserción de registros

Se registraron cinco productos de ejemplo, distribuidos entre las distintas categorías que maneja la tiendita, cada uno con su precio y su cantidad en existencia.

```
# INSERT: alta de productos
run("""INSERT INTO productos (nombre, tipo, precio, existencia) VALUES
    ('Papas fritas', 'snack', 15.00, 50),
    ('Refresco', 'bebida', 18.00, 40),
    ('Chocolate', 'dulce', 12.00, 30),
    ('Cuaderno', 'papelería', 25.00, 20),
    ('Lápiz', 'papelería', 5.00, 100);""")

query("SELECT * FROM productos")
```

OK -> INSERT INTO productos (nombre, tipo, precio, existencia) VALUES ...

	id_producto	nombre	tipo	precio	existencia
0	1	Papas fritas	snack	15	50
1	2	Refresco	bebida	18	40
2	3	Chocolate	dulce	12	30
3	4	Cuaderno	papelería	25	20
4	5	Lápiz	papelería	5	100

Figura 14. Inserción de los productos en la tabla productos.

Modificación de registros

Se aumentó un diez por ciento al precio de los productos de la categoría papelería, como ejemplo de actualización aplicada a un grupo de productos.

```
[12]
✓ 0 s
# UPDATE: aumento de precio a la categoría "papelería"
run("UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'papelería';")
query("SELECT id_producto, nombre, tipo, precio FROM productos WHERE tipo = 'papelería'")

OK -> UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'papelería'; ...

  id_producto  nombre  tipo  precio
0           4 Cuaderno papelería  27.5
1           5  Lápiz papelería   5.5
```

Figura 15. Actualización del precio de los productos de la categoría papelería.

Eliminación de registros

Por último, se eliminó de la tabla el producto con menor cantidad en existencia, simulando el caso en que un producto se da de baja del inventario.

```
[13]
✓ 0 s
# DELETE: eliminación de registros (se retira el producto con menor existencia)
run("DELETE FROM productos WHERE existencia < 25;")
query("SELECT * FROM productos")

... OK -> DELETE FROM productos WHERE existencia < 25; ...

  id_producto  nombre  tipo  precio  existencia
0           1 Papas fritas  snack  15.0      50
1           2  Refresco  bebida  18.0      40
2           3  Chocolate  dulce  12.0      30
3           5  Lápiz papelería  5.5     100
```

Figura 16. Eliminación del producto con menor existencia.

Conclusiones

A través de estos tres ejercicios se practicaron las operaciones básicas para trabajar con una base de datos: la creación de tablas definiendo correctamente el tipo de dato y las restricciones de cada campo, la inserción de información nueva, la modificación de registros existentes bajo una condición específica y la eliminación de aquellos que ya no eran necesarios. Aunque los tres casos trataron temas distintos —un restaurante, una escuela y una tiendita escolar—, en todos se siguió la misma lógica de trabajo, lo cual ayudó a comprender que estas cuatro operaciones (CREATE, INSERT, UPDATE y DELETE) son la base para administrar cualquier tipo de información dentro de una base de datos relacional. Además, se reforzó la importancia de usar condiciones claras en las instrucciones UPDATE y DELETE, ya que de no especificarse correctamente podrían afectar más registros de los deseados.